Developing a diagnostic reasoning model for root-cause identification in industrial printing systems

Ishaan Almeida
2120348
Department of Mathematics and Computer Science
Eindhoven University of Technology, Netherlands

***Abstract*—Industrial printers such as the Canon Production Printing Colorado M-series generate broad error codes and textual descriptions that often do not uniquely identify the underlying root cause. As a result, technicians must perform diagnostic actions and interpret their outcomes to narrow down plausible root causes. This can lead to long diagnosis times and limited reuse of knowledge across cases when observations and outcomes are recorded as free text rather than as consistent structured labels. This preparation report proposes a data-guided probabilistic diagnostic reasoner for root cause identification and describes the iterative procedure it follows during a diagnostic session: (i) rank plausible root causes from the observations available at the start of a session, (ii) update posterior probabilities as diagnostic action outcomes are observed and monitor uncertainty over root cause hypotheses using entropy, and (iii) recommend the next diagnostic action using an explicit trade-off between expected uncertainty after the action and a domain-defined diagnostic cost. The report also states the research questions, summarizes related work, and presents an execution plan and risk assessment for implementing and evaluating the reasoner on historical diagnostic cases. A grow-with-data evaluation is included to assess whether a richer model that relaxes the baseline independence assumption improves ranking performance as structured session logs accumulate.**

I. Problem statement

Canon Production Printing (CPP) currently performs diagnostic tasks for the Colorado M-series printers through a largely manual workflow. When a fault occurs, the printer reports error codes and broad error descriptions. These observations often do not uniquely identify the underlying root cause, because similar symptoms may arise from different component failures or operating conditions. As a result, identifying the root cause typically requires a diagnostic session in which additional observations are collected and the set of plausible root causes is narrowed over time.

A. General problem

Printer error codes and technician observations during diagnosis often do not uniquely identify the underlying root cause, as the same symptoms may arise from different root causes.

This motivates an objective that ranks plausible root causes from initial observations and refines this ranking using diagnostic action outcomes during the session. In practice, the information available before a diagnostic session typically consists of printer-reported observations and, in some cases, a customer description of the issue (often free text). This information can support an initial narrowing of plausible root causes, but it is often not sufficiently specific to identify the true cause with confidence. Technicians therefore execute diagnostic actions during the session and use their outcomes as additional observations to refine their root cause hypotheses until the issue is resolved or the case is escalated.

This process can be time-consuming. Because early observations are ambiguous, technicians may need to execute multiple diagnostic actions before converging on a root cause. This can lead to long diagnosis times, repeated service visits, and in some cases unnecessary replacement of expensive components when an early hypothesis is incorrect.

A further limitation is that diagnostic observations gathered during these sessions are not consistently captured in a structured form. In the current workflow, technicians log diagnostic actions and their outcomes as free-text notes rather than as predefined fields (e.g., action name and outcome label). This makes records inconsistent and difficult to interpret reliably, which limits the reuse of past cases for evaluation and future reference. In addition, case termination (e.g., whether a case was resolved or escalated) may be recorded implicitly in free text rather than as an explicit label, which further reduces reliability for retrospective analysis.

This work therefore explores a data-guided diagnostic reasoner that (i) ranks plausible root causes from the observations available at the start of a diagnostic session, (ii) refines this ranking using diagnostic action outcomes observed during the session, and (iii) defines structured logging of actions and outcomes to enable consistent reuse of historical cases for evaluation and later refinement. To make this feasible under CPP's current data constraints, the thesis first establishes (i) a structured case schema and outcome label design as a prerequisite for consistent retrospective evaluation. Building on this foundation, the thesis contributes (ii) a method to map free-text descriptions into a predefined symptom indicator set so the same reasoning model can operate on consistent inputs, and (iii) a baseline probabilistic diagnostic reasoner evaluated under CPP's current data constraints for ranking, posterior updating, and cost-aware diagnostic action selection. Finally, once sufficient structured session logs are accumulated, the thesis evaluates whether a richer model that relaxes the baseline independence assumption improves ranking performance over the initial reasoner. Figure 1 describes how the proposed reasoner operates during a diagnostic session.

II. Related work

This section reviews prior work that is directly relevant to the challenges in CPP's diagnostic setting. The literature is used to (i) position the work relative to existing industrial fault diagnosis frameworks, (ii) clarify how root-cause identification is typically modeled under ambiguous observations, (iii) show how diagnosis can be refined step-by-step using outcomes of diagnostic actions, (iv) summarize how diagnostic actions can be selected when actions have different costs, and (v) identify the practical role of structured logging in enabling retrospective evaluation and model improvement. The goal is to identify what is well-studied and what remains less developed for field-service scenarios where observations are broad and historical records are not consistently structured.

Throughout this report, *diagnostic actions* refer to any performative actions, checks and tests, *outcomes* refer to the observed result of a diagnostic action, and *root cause hypotheses* refer to candidate root cause labels.

A. Industrial fault diagnosis: FMEA and case-based reasoning

Two lines of work from industrial fault diagnosis are relevant to CPP's setting before considering the probabilistic literature. Failure Mode and Effects Analysis (FMEA) is a standard industrial framework that systematically maps component failure modes to their observable effects and to recommended diagnostic or corrective actions, with associated severity and occurrence ratings [1]. The structure of FMEA documentation - linking failure modes, observable symptoms, and remediation actions, is conceptually close to the knowledge base structure used in diagnostic reasoners of the kind proposed here. However, FMEA does not provide a probabilistic inference mechanism for ranking root causes under ambiguous observations; it is a design-time documentation tool rather than a runtime reasoner, and it does not account for the uncertainty inherent in broad, overlapping error codes.

Case-Based Reasoning (CBR) offers a complementary perspective: rather than reasoning from a generative probabilistic model, CBR retrieves historically similar cases and adapts their solutions to the current problem [2]. CBR is directly relevant to the reuse of historical diagnostic sessions proposed in this thesis. However, reliable case retrieval requires consistently structured records, which is precisely the limitation that the structured logging component of this work addresses as a prerequisite. This motivates addressing the structured logging limitation first, before case retrieval methods can be reliably applied.

B. Probabilistic diagnosis and root-cause ranking under ambiguous observations

A foundational line of research in model-based diagnosis represents a system using *behavioral models*, which are simplified descriptions of how components are expected to behave under normal operation. When observed behavior deviates from these expectations, the diagnosis task is framed as selecting a set of *fault hypotheses* that could explain the observations. In this report, a fault hypothesis corresponds to a *root cause hypothesis*. A set of *plausible root causes* refers to the subset of root cause hypotheses that remain reasonable given the currently available observations.

The foundations for probabilistic reasoning over hypothesis spaces are established in Pearl's framework of belief networks and probabilistic inference [4]. Building on this, de Kleer and Williams introduced probabilistic reasoning over multiple-fault hypotheses, including ranking candidates using prior probabilities and updating posterior probabilities as new observations are collected [3]. This provides the conceptual basis for treating root-cause identification as inference over a hypothesis space, which is important when initial observations do not uniquely determine the cause.

Ranked outputs are often more practical than single-label predictions in settings where early observations are broad. When the same error code or symptom pattern can be produced by multiple root causes, the first available observations may only reduce the candidate set rather than identify one cause with certainty. Rank-based approaches have also been studied in active diagnosis, where additional queries or diagnostic actions are selected to identify the underlying state efficiently [5]. However, much of this literature assumes abstract diagnostic actions with well-defined response models and structured observations [5].

In the CPP setting, initial observations consist of broad error codes and error descriptions, together with structured machine warnings and lifetime counters. To use these observations for diagnosis, the knowledge base must include explicit links between error codes/warnings/counters and the root causes they can indicate. This motivates evaluating whether probabilistic ranking improves diagnostic usefulness compared to non-probabilistic baselines derived from existing documentation. In practice, technicians typically consult error code documentation to obtain a candidate list of plausible root causes, optionally following a fixed priority order when such guidance exists. This represents the current manual workflow and serves as a natural non-probabilistic comparator for any proposed method.

C. Sequential diagnosis: posterior updating using diagnostic action outcomes and uncertainty tracking

In this section, the term hypothesis refers to a *root cause hypothesis*. Sequential diagnosis frameworks model diagnosis as an iterative process in which each new observation updates the posterior probabilities over root cause hypotheses. Siddiqi and Huang propose sequential diagnosis by abstraction, showing that structuring the hypothesis space can make sequential updates feasible at scale [6]. Feldman et al. propose a model-based active testing framework that selects diagnostic actions to reduce uncertainty over root cause hypotheses [7]. These works align with field diagnosis in the sense that technicians perform diagnostic actions and interpret their outcomes as additional observations.

A key methodological idea is that diagnostic action outcomes should be treated as new observations that update the posterior probabilities over root cause hypotheses. Uncertainty over root causes can be quantified using Shannon entropy [8].

When entropy decreases across diagnostic actions, the posterior distribution becomes more concentrated, meaning fewer root causes remain plausible under the model. While entropy-based objectives appear in active testing and sequential diagnosis, many studies assume that observations, diagnostic actions, and outcomes are already available as structured variables in their datasets or benchmarks. This differs from CPP's setting where diagnostic actions and outcomes are often recorded as free text, which motivates explicitly defining structured outcome labels for retrospective analysis.

D. Cost-aware and uncertainty-aware action selection

Decision-theoretic diagnosis and active diagnosis study how to choose diagnostic actions that improve diagnostic accuracy while accounting for the cost of performing those actions. Prior work considers cost-aware strategies in sequential diagnosis, including adaptive probing under probabilistic dependencies [9], optimization-based diagnosis with dependent actions [10], and POMDP-based strategies for sequential diagnostic planning with diagnostic actions that can have noisy outcomes [11]. A common theme is the trade-off between the diagnostic value of an action and the resources required to execute it.

Several studies link this trade-off to the expected entropy reduction. Zheng et al. study active diagnosis with diagnostic action costs and use conditional entropy to guide sequential diagnostic action selection to reduce uncertainty under cost constraints [12]. More recently, van Gerwen et al. propose a cost-aware active diagnosis approach for differential diagnosis that optimizes a combination of diagnostic uncertainty (entropy) and action cost, arguing that such a combined objective can be preferable to simple "expected entropy reduction per cost" heuristics [13].

A practical difference in the CPP context is that action costs and action outcome probabilities are not always available as reliable numeric labels in historical records. This motivates starting with domain-defined cost categories (time, effort, parts) and using structured action and outcome labels to support retrospective evaluation and later refinement.

E. Structured logging as an enabler for reuse and improvement

A practical challenge in field diagnosis is that diagnostic actions and outcomes are often recorded as free-text notes, which makes aggregation and comparison across cases difficult. Prior work on log parsing shows that free-form system logs can be converted into structured templates that enable downstream analysis. For example, He et al. propose Drain, an online log parsing approach that extracts structured event templates from repetitive, machine-generated log entries [14]. While Drain targets structured system logs rather than natural language text, the broader motivation applies here: converting unstructured records into consistent structured variables is a prerequisite for aggregation, comparison, and model evaluation across cases.

In CPP's setting, two related structuring steps are required. First, defining a structured schema for diagnostic actions and outcomes (e.g., action name and outcome label) so that historical cases can be reused consistently for retrospective analysis and later model refinement. Second, mapping free-text customer descriptions and technician notes into a predefined set of symptom indicators so that downstream diagnostic reasoning operates on consistent structured variables. This second step is a natural language understanding task: unlike templated system logs, customer descriptions exhibit semantic variability and require a classifier trained to detect symptom keywords from a predefined catalogue, rather than a template extraction approach [15].

F. Evaluation metrics for ranked root-cause identification

When the output of a diagnostic method is a ranked list of root causes, evaluation is phrased in terms of where the ground truth root cause appears in that ranked list. A common metric is *mean rank*, which averages the rank position of the ground truth root cause across cases (lower is better). Another common metric family is *top-k inclusion* (Hit@k / Top-k accuracy), which measures the fraction of cases where the ground truth root cause appears within the top k ranked candidates (higher is better).

Rank-based evaluation is also common in diagnosis settings where systems output a shortlist of candidate causes. For example, symptom checker systems are evaluated based on whether the correct diagnosis appears first or within a top-k list of suggestions [16].

These metrics are used in this thesis because the practical goal under broad initial observations is to produce a short candidate list that still contains the correct root cause. Mean rank captures how many candidates must be checked on average, while top-k inclusion captures whether the correct cause appears in a shortlist of size k that can be inspected during diagnosis.

G. Summary and research gap

Prior work provides industrial fault diagnosis frameworks in FMEA and CBR [1], [2], foundations for probabilistic diagnosis and hypothesis ranking [3], [4], sequential posterior updating and active testing [6], [7], entropy-based uncertainty measurement [8], and cost-aware action selection [9], [10], [11], [12], [13]. However, many approaches are evaluated in settings where observations and outcomes are already structured and where action outcome probabilities and costs are cleanly specified.

CPP's setting differs in three practical ways. First, initial observations are broad error codes and error descriptions, so ambiguity is common and ranking multiple plausible root causes is often more realistic than a single prediction. Second, diagnosis proceeds through diagnostic actions whose outcomes should be used as additional observations to refine posterior probabilities across the session. Third, historical technician records are often free text, which limits reuse unless diagnostic actions and outcomes are mapped into consistent structured labels. These gaps motivate studying diagnostic ranking, posterior updating, and action selection using a data-guided approach that is feasible under CPP's current data constraints and can be refined as more structured records become available.

A fourth gap, not addressed by existing work in field-service settings, is whether the conditional independence assumption underlying the baseline probabilistic reasoner can be relaxed as structured session logs accumulate, and whether doing so measurably improves ranking performance. The proposed approach addresses this through an explicit grow-with-data evaluation that tests a richer model once sufficient structured cases have been logged, without requiring a change to the core problem formulation or evaluation metrics.

III. Research questions

A. RQ1: Root-cause ranking from non-specific initial observations

Does a probabilistic diagnostic reasoner produce better mean rank and top-k inclusion of the ground truth root cause than non-probabilistic baselines (knowledge base lookup and heuristic scoring), when using non-specific initial observations available at the start of a diagnostic session?

B. RQ1.1: Effect of free-text symptom mapping on root-cause ranking

How does adding symptom indicators that are mapped from free-text descriptions affect mean rank and top-k inclusion, compared to using only printer-reported observations and other structured machine data?

C. RQ2: Posterior probability and entropy across diagnostic actions

When diagnostic action outcomes are incorporated as new observations in the posterior update over root cause hypotheses, does (i) the posterior probability of the ground truth root cause increase on average across diagnostic actions and (ii) Shannon entropy over root cause hypotheses decrease on average across diagnostic actions, compared to the initial posterior computed from non-specific observations alone?

D. RQ3: Uncertainty-aware action selection under diagnostic cost

Does uncertainty-aware action selection reduce the cumulative diagnostic cost *until session termination* (resolution or hypotheses exhaustion), compared to baseline action selection strategies?

E. RQ4: Ranking improvement under a richer model using accumulated structured logs

Once sufficient structured session logs have been accumulated, does a model that relaxes the conditional independence assumption of the Naive Bayes baseline, such as a Bayesian network with learned dependencies between observations, improve mean rank and top-k inclusion of the ground truth root cause on held-out cases, compared to the baseline probabilistic reasoner?

IV. Proposed approach

A. High-level idea

This report proposes a probabilistic diagnostic reasoner that follows the iterative procedure in Fig. 1. The reasoner first computes posterior probabilities over root cause hypotheses using the observations available at the start of a diagnostic session (initial observations). It then enters an iterative loop in which it recommends a diagnostic action, observes the action outcome, and updates the posterior probabilities by treating the outcome as an additional observation. This loop continues until the issue is resolved or the case is escalated (e.g., because the available observations do not support any root cause hypothesis in the knowledge base). All observations, diagnostic actions, and outcomes are logged in a structured format and used for offline parameter updates.

The approach is designed to be implementable with the currently available historical cases, while keeping the model structure compatible with later refinement when more structured diagnostic sessions become available. The report does not claim that the action selection policy is optimal; instead, it evaluates a simple baseline that selects the next diagnostic action by comparing the expected uncertainty after that action together with its diagnostic cost. This baseline can be improved as stronger outcome probability estimates and more complete action/outcome traces become available.

B. Inputs, outputs, and knowledge base

A diagnostic session is defined at the level of a single case with one ground truth root cause label. A root cause label refers to either a specific component failure or an operating condition that is treated as the primary cause for the case.

Inputs. Before the first diagnostic action, the reasoner receives printer-reported observations, including one or more error codes and their textual descriptions. Each error code has a textual description, but the same error code can still correspond to multiple plausible root causes. The reasoner also receives current machine data from the printer, including machine warnings (available as structured fields) and lifetime counters. In addition, the case may include a customer description of the issue, which is often free text.

A key practical challenge is that parts of the initial observations are not available as structured variables in historical records. In particular, customer descriptions and technician notes often contain symptom information as free text. To use such cases consistently, free text is mapped to a predefined set of structured variables (e.g., symptom indicators and error code references) when possible.

This mapping is performed as a multi-label classification task: given a free-text description, the classifier predicts which symptom indicators from the predefined catalogue are present. As a baseline, a TF-IDF feature representation combined with a one-vs-rest logistic regression classifier provides an interpretable and computationally tractable starting point that is well-suited to the expected label set size and available data volume [15]. If the labelled case volume supports it, a fine-tuned

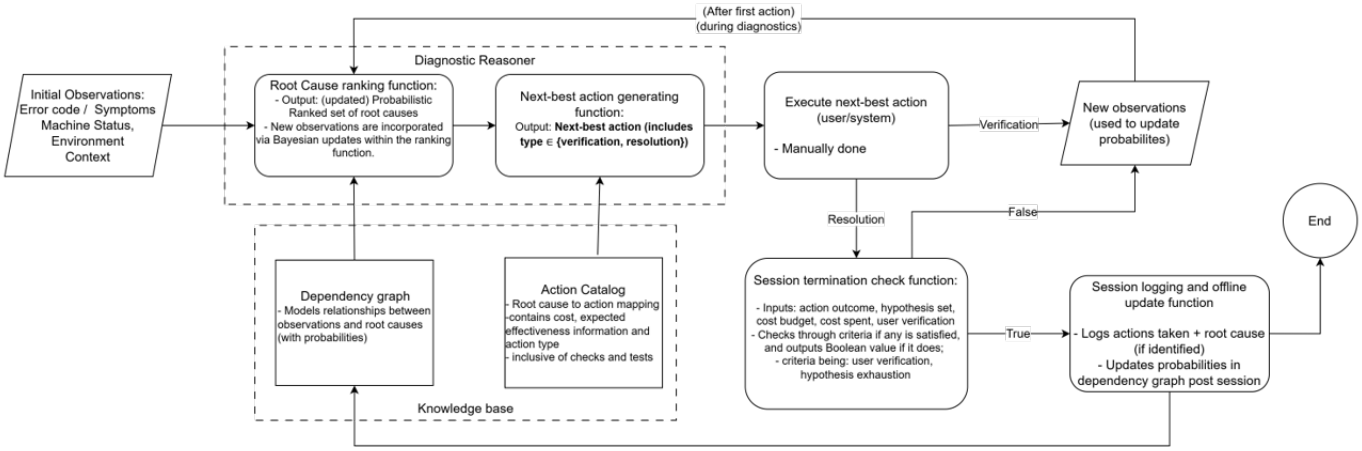

Fig. 1. Procedure followed by the proposed diagnostic reasoner: initial observation ingestion, action recommendation, outcome observation, posterior update, session termination, and structured logging with parameter updates.

sentence embedding model will be evaluated as an alternative. The reasoner operates on the resulting structured symptom indicators.

To evaluate whether this mapping is reliable enough for the diagnostic reasoner, the classifier is evaluated on held-out historical cases where symptom indicators can be derived, using precision, recall, and F1-score (reported per symptom indicator and as overall averages). In addition, the thesis evaluates the contribution of the free-text mapping step for RQ1.1 by comparing the ranking performance with and without mapped symptom indicators. This isolates the impact of the free-text mapping step on mean rank and top-k inclusion.

During the diagnostic session, the reasoner observes a sequence of executed diagnostic actions and their outcomes. Outcomes are represented as binary or categorical variables based on predefined outcome labels for each action.

Outputs. The reasoner outputs (i) a ranked list of root cause labels with associated posterior probabilities and (ii) a recommended next diagnostic action. It also produces a structured session log containing the initial observations, the executed action sequence, observed outcomes, and the session termination status (e.g., resolved or escalated).

Knowledge base and representation. The reasoner relies on (i) a probabilistic dependency graph that relates observations (error codes/descriptions, machine warnings, lifetime counter ranges, symptom indicators, and action outcomes) to root cause labels, and (ii) an action catalog that enumerates available diagnostic actions, their possible outcomes, and their diagnostic intent (verification vs. resolution). Machine warnings are already recorded as structured fields. Lifetime counters are numeric and are represented as domain-defined categorical ranges (e.g., low/medium/high) so they can be used as discrete observations in the inference model.

Cost representation. Diagnostic action costs are predefined using domain knowledge for technician time, technician effort, and part replacement. Each cost attribute is mapped to an ordinal value (low $= 1$, medium $= 2$, high $= 3$). A single scalar cost score is computed as a weighted sum:

$$c(a) = w_T\, c_T(a) + w_E\, c_E(a) + w_P\, c_P(a),$$

where $c_T(a)$, $c_E(a)$, and $c_P(a)$ are the ordinal values for time, effort, and parts for action a. As a starting point, weights are set uniformly ($w_T = w_E = w_P = 1$) and can be adjusted using weekly feedback. The resulting cost score is used for comparing action selection strategies rather than as a calibrated time or monetary estimate.

Scope note. This thesis focuses on defining the structured fields required by the reasoner and using them for retrospective evaluation on historical cases. Deploying new user-interface forms for data entry is out of scope.

C. Probabilistic root-cause ranking

Objective. Given the initial observation set O_0 (before any diagnostic action outcome is observed), compute posterior probabilities over root cause hypotheses and produce a ranking:

$$P(R \mid O_0).$$

Root causes are ranked in descending order of $P(R = r \mid O_0)$. This corresponds to the first iteration of Fig. 1.

Approach / formulation. A probabilistic dependency model is defined over a discrete root cause variable R and a set of discrete observation variables $O = \{O_i\}$ (e.g., error codes/descriptions, machine warnings, lifetime counter ranges, and symptom indicators). The model consists of a prior distribution $P(R)$ and conditional probability tables that link root causes to observations (e.g., $P(O_i \mid R)$), consistent with probabilistic reasoning over hypothesis spaces [4], [3]. Under the conditional independence assumption introduced below, this corresponds to a Naive Bayes classifier over the root cause variable, a model known to remain effective for ranking tasks even when the independence assumption is violated in practice

[17]. Because the model defines both $P(R)$ and $P(O_0 \mid R)$, the posterior $P(R \mid O_0)$ follows directly from Bayes' rule:

$$P(R = r \mid O_0) = \frac{P(O_0 \mid R = r)\, P(R = r)}{\sum_{r' \in \mathcal{R}} P(O_0 \mid R = r')\, P(R = r')}.$$

The likelihood term $P(O_0 \mid R = r)$ is obtained from the dependency model. Under the Naive Bayes baseline, observation variables are treated as conditionally independent given the root cause, so the likelihood factorizes as

$$P(O_0 \mid R = r) = \prod_i P(O_i = o_i \mid R = r),$$

where o_i is the observed value of variable O_i. Model parameters are initialized using domain knowledge and knowledge base links, and are refined using frequency estimates from historical cases when sufficient data is available.

Evaluation and baselines. Ranking quality is evaluated using mean rank and top-k inclusion of the ground truth root cause across historical cases (as discussed in the evaluation subsection of Related Work). Results are compared against two non-probabilistic baselines that do not compute posterior probabilities.

First, a *knowledge base lookup baseline* simulates the current manual technician workflow described in Section II, using the priority order from documentation where available.

Second, a *heuristic scoring baseline* assigns a simple score to each candidate root cause based on matched observations and ranks root causes by this score. A starting point is a weighted match count:

$$s(r) = \sum_i w_i\, m_i(r), \tag{1}$$

where $w_i \geq 0$ allows emphasizing certain observation types (e.g., error codes vs. symptom indicators), and $m_i(r) \in \{0, 1\}$ equals 1 if the knowledge base contains an explicit link between root cause r and observation type O_i (e.g., r can produce that error code, warning, counter range, or symptom indicator), and 0 otherwise. This baseline combines multiple observations using transparent rules, without probabilistic updating.

These baselines are chosen because they provide clear non-probabilistic comparators that reflect how diagnostic information can be used without a probabilistic model: (i) direct lookup from documented mappings and (ii) simple rule-based aggregation across multiple observations.

D. Probability updating with diagnostic action outcomes

Objective. During diagnosis, each observed diagnostic action outcome is treated as an additional observation that refines posterior probabilities over root cause hypotheses. Let O_t denote the observations available up to diagnostic action t, and let o_t denote the observed outcome of the executed diagnostic action at time t. The update is:

$$P(R \mid O_t) \rightarrow P(R \mid O_t \cup \{o_t\}).$$

Conditional independence assumption. As a starting point, observation variables are treated as conditionally independent given the root cause under the Naive Bayes baseline, which makes parameter estimation feasible under limited structured data. This assumption may be violated in practice (e.g., correlated symptoms) and is evaluated explicitly in RQ4 once sufficient structured logs are available.

What is measured. After each diagnostic action outcome is observed, the reasoner recomputes the posterior distribution $P(R \mid O_t)$. Two quantities are tracked along the diagnostic action sequence. First, the posterior probability assigned to the ground truth root cause label R:

$$P(R = R^* \mid O_t).$$

This value is the probability mass assigned by the updated posterior to the known ground truth label, and is used to track whether evidence increases support for the correct root cause. Second, uncertainty over root causes measured by Shannon entropy:

$$H(P(R \mid O_t)) = -\sum_r P(r \mid O_t) \log P(r \mid O_t).$$

These measures indicate whether diagnostic action outcomes increase probability mass on the ground truth root cause and reduce uncertainty across diagnostic actions [8]. The log base is not critical here because entropy is used to compare uncertainty trends across diagnostic actions.

Design choice and alternatives. This sequential updating design follows established ideas in probabilistic diagnosis and active testing [3], [7], while remaining implementable under CPP's current data constraints. More expressive alternatives are possible, such as dependency models that explicitly represent correlations between observations (beyond the Naive Bayes baseline), which is evaluated as structured logs accumulate (RQ4). For action selection specifically, a POMDP-based policy represents the principled next step beyond the entropy-plus-cost heuristic: a POMDP jointly optimises diagnostic action sequences by treating the root cause as a hidden state and action outcomes as observations, but requires reliable transition and observation probability estimates that are only estimable once sufficient structured session logs are available [11]. This is therefore identified as a concrete future direction rather than a thesis objective.

E. Next-best diagnostic action selection with expected entropy and cost

At each iteration, the reasoner selects a diagnostic action that is expected to reduce uncertainty over root causes while accounting for diagnostic cost. Each candidate diagnostic action is evaluated based on the expected uncertainty after observing its outcome, and the action with the best trade-off between expected uncertainty and cost is selected.

For a candidate action a, let $\mathcal{O}(a)$ denote its possible outcomes. The expected posterior entropy over root causes after executing a is:

$$E[H_R \mid a] = \sum_{o \in \mathcal{O}(a)} P(o \mid O_t, a)\; H(P(R \mid O_t, o))\,.$$

Each diagnostic action a has a fixed, domain-defined scalar cost $c(a)$ as defined earlier. The next-best action is selected by minimizing a combined objective:

$$a_{t+1} = \arg\min_{a} \Big(E[H_R \mid a] + \lambda\, c(a) \Big).$$

Here, $\lambda \geq 0$ controls the trade-off between diagnostic uncertainty reduction and diagnostic cost. This objective is used as a practical cost–uncertainty trade-off rule rather than a claim of optimal planning. In evaluation, λ is selected using a simple sensitivity analysis on a validation split (reporting results for a small set of reasonable values).

Outcome probability estimation. Outcome probabilities $P(o \mid O_t, a)$ are estimated from historical cases when enough samples exist for stable estimates. When data are too sparse, domain-informed initial probabilities are used as a modelling assumption and refined as more structured session logs become available.

Baseline action selection strategies include a fixed-order action list baseline (documentation order), a cheapest-first baseline based on the domain-defined cost categories, an uncertainty-only greedy baseline (minimize $E[H_R \mid a]$), and a random applicable-action baseline.

Cumulative cost logging. For each session, cumulative diagnostic cost after each diagnostic action is computed:

$$C_t = \sum_{i=1}^{t} c(a_i),$$

where $c(a_i)$ is the scalar cost of diagnostic action a_i. For comparison across strategies, cumulative cost until session termination is reported, and additionally cost-to-resolution for resolved sessions and cost-to-escalation for escalated sessions.

F. Session termination logic

The session termination logic follows Fig. 1. If a resolution action is performed and the issue is resolved, the session terminates. If the issue is not resolved, the action outcome is treated as an observation and the session continues. The session terminates as escalated when the root cause hypotheses are exhausted or when the case is closed without resolution.

In historical cases, termination status is taken from the case's *error resolution* field, which records what happened: *resolved* indicates the fix was confirmed to work and the diagnostic session was closed, while *escalated* indicates the session was closed without resolution and the issue was escalated. This field is used only as a label for retrospective evaluation and does not imply that resolution is dependent on the field itself.

The hypothesis set at diagnostic action step t is defined as the set of plausible root causes under the current posterior:

$$\mathcal{H}_t = \{r \in \mathcal{R} \;:\; P(R = r \mid O_t) \geq \tau\},$$

where $\mathcal{R}$ is the set of root causes represented in the knowledge base and τ is a plausibility threshold. Hypotheses exhaustion means that no modeled root cause remains sufficiently plausible (i.e., none exceed τ), which can indicate that the true cause is missing from the knowledge base or that the observation mapping is incomplete. The threshold τ is set using domain knowledge and examined through sensitivity checks to ensure conclusions are not driven by a single arbitrary value.

G. Structured logging and offline update

For each session, the reasoner logs the initial observations (including machine warnings, lifetime counter ranges, and symptom indicators derived from free text where needed), the executed diagnostic action sequence, and the observed outcomes. The ground truth root cause label is associated with the session for evaluation.

Parameter update design. Two types of offline parameter updates are applied without changing the model structure. First, the prior $P(R)$ is updated after every completed diagnostic session using an incremental frequency count over observed root cause labels. This update is lightweight and stable because it is a simple count-based operation over a single new observation. Second, the conditional probability tables $P(O_i \mid R)$ and action outcome likelihoods are updated periodically in batches, using frequency estimates with Laplace smoothing. Batching these updates avoids instability that could arise from updating CPTs on a single noisy session, and Laplace smoothing ensures that zero-count entries do not collapse probabilities for unobserved combinations. Both update types are design decisions justified by their computational tractability and stability properties under CPP's sparse data constraints, and do not constitute a separate empirical evaluation within the thesis.

Grow-with-data evaluation (RQ4). As structured session logs accumulate over the thesis period, the conditional independence assumption underlying the Naive Bayes baseline becomes increasingly testable. Once sufficient structured cases are available, the thesis evaluates whether a richer model that relaxes this assumption - such as a Bayesian network with learned dependencies between observations - improves mean rank and top-k inclusion on held-out cases compared to the Naive Bayes baseline. This evaluation uses the same metrics and data splits defined for RQ1, applied to the accumulated structured logs rather than the initial historical dataset.

V. Project planning

This section outlines the execution plan to complete the thesis work after the preparation report. The plan is organised to first make historical cases usable, then build an evaluation framework and baselines, and only then implement and evaluate the probabilistic diagnostic reasoner. Weekly feedback with domain experts will be used to validate mappings, outcome labels, and the cost table.

A. Milestones and tasks

M1: Structured schema and logging design (foundation). The first milestone is to define what a "case" and a "diagnostic session" look like in a consistent format so that historical cases can be reused reliably.

- Define a single case schema for observations, diagnostic actions, outcomes, termination status, and root cause labels.

- Define the structured logging format (field names and allowed values), consistent with the schema.
- Define the label set for symptoms and action outcomes (used later for retrospective mapping and evaluation).

M2: Structured dataset and evaluation framework (before methods). This milestone makes the data usable and defines how results will be measured.

- Structure a first ink-system subset of historical cases into the schema (including resolved vs. escalated termination status when available).
- Implement the evaluation framework: data splits, evaluation scripts, and metrics (mean rank and top-k inclusion).
- Implement the NLP classifier pipeline that maps free-text descriptions to predefined error/symptom indicators, and evaluate it on held-out labelled cases using precision/recall/F1.

M3: Baselines (non-probabilistic comparators). This milestone implements baselines that do not compute posterior probabilities.

- Implement the knowledge base lookup baseline (candidate root causes linked to observed error code(s), with documented priority order if available).
- Implement the heuristic scoring baseline (transparent match-based scoring over observations).
- Run baseline evaluations using the evaluation framework.

M4: Probabilistic root-cause ranking and posterior updating (RQ1–RQ2). This milestone implements the probabilistic dependency model and evaluates ranking and updating.

- Implement posterior computation for initial observations $P(R \mid O_0)$ and evaluate ranking (mean rank, top-k inclusion).
- Implement posterior updating using diagnostic action outcomes and compute $P(R = R^* \mid O_t)$ and $H(P(R \mid O_t))$ across diagnostic actions.
- Summarise trends across cases (when probabilities increase and when entropy decreases).

M5: Diagnostic action selection and cost comparison (RQ3). This milestone implements uncertainty-aware action selection and compares strategies using cumulative cost until termination.

- Implement action selection using expected posterior entropy after a diagnostic action with a cost penalty.
- Validate the ordinal cost table (time/effort/parts) and the scalar cost mapping used for comparisons.
- Compare strategies using cumulative cost until termination; report cost-to-resolution, cost-to-escalation, and escalation rate.

M6: Grow-with-data evaluation and thesis writing (RQ4). This milestone consolidates the work into a complete thesis report and evaluates the grow-with-data model improvement using accumulated structured logs.

- Verify that sufficient structured session logs have been accumulated during M1–M5 to support evaluation of a richer model.
- Implement a Bayesian network with learned dependencies between observations as an alternative to the Naive Bayes baseline, relaxing the conditional independence assumption.
- Evaluate whether the richer model improves mean rank and top-k inclusion on held-out cases compared to the Naive Bayes baseline (RQ4), using the same evaluation framework defined in M2.
- Write the thesis after results are available, starting from the back: discussion and results first, then methods, then introduction and related work updates.
- Final revision and submission preparation.

B. Timeline and concurrency

The timeline uses Week 1 as the first week after the preparation report submission. As shown in Fig. 2, the work starts with schema/logging design and data structuring, because all later evaluation depends on consistent labels and fields. The evaluation framework is implemented before the baselines and the probabilistic method to ensure that comparisons are reproducible. Thesis writing is planned after results are available, starting with results and discussion, followed by methods and then the introduction and final edits.

VI. Risks

This section lists the main risks that could affect the plan, together with likelihood, prevention, and what will be done if the risk materialises.

A. Historical cases may take longer to structure

Likelihood. Medium–High.
Prevention. Start with a strict ink-system subset and lock a minimum viable schema early (M1).
If it happens. Reduce scope to the most complete cases first (root cause label + at least partial action outcomes), report coverage explicitly, and keep the initial model simple.

B. Free-text mapping quality may be insufficient

Likelihood. Medium.
Prevention. Use a constrained label set (predefined error/symptom indicators) and validate the NLP classifier on a held-out set early in M2.
If it happens. Fall back to a rule-based keyword baseline for the mapping step, and restrict evaluation to cases where structured labels can be derived reliably.

C. Outcome data may be incomplete or inconsistent

Likelihood. Medium.
Prevention. Define outcome label sets per diagnostic action and validate them with domain experts during M1–M2.
If it happens. Treat missing outcomes as missing (no imputation), restrict RQ2/RQ3 evaluation to action segments with usable outcomes, and report how much data is excluded.

D. Knowledge base coverage gaps

Likelihood. Medium.
Prevention. Validate mappings for the ink system first and document known coverage limits.
If it happens. Treat hypotheses exhaustion and escalation as valid outcomes, report escalation rate, and use escalated cases to identify missing mappings or missing root causes for later iterations.

E. Knowledge base construction quality

Likelihood. Medium–High.
Prevention. Validate all dependency links between error codes, symptom indicators, actions, and root causes against domain expert knowledge during M1, and document the rationale and confidence level for each link explicitly.
If it happens. Incorrect or inconsistent links in the knowledge base will propagate errors to all downstream components, including ranking, posterior updating, and action selection. In this case, evaluation is restricted to the subset of root causes whose knowledge base links have been validated with domain experts, and the coverage limitation is reported explicitly as a boundary condition on the results.

F. Cost table may be debated

Likelihood. Medium.
Prevention. Validate the ordinal cost table early (time/effort/parts) through weekly feedback and document the rationale.
If it happens. Report results under two reasonable cost tables and include a sensitivity check to show whether conclusions depend on small changes.

G. Retrospective evaluation limits

Likelihood. Medium.
Prevention. Clearly define what is simulated retrospectively (action selection) and what is observed from records (actions taken, outcomes, termination status).
If it happens. Keep conclusions limited to what retrospective simulation supports, report where simulation assumptions differ from recorded traces, and frame results as expected gains under the model.

H. Time pressure as a single contributor

Likelihood. Medium.
Prevention. Freeze scope (ink system first) and keep weekly checkpoints with domain experts to avoid rework.
If it happens. Prioritise completing M2–M5 (structured data, baselines, RQ1–RQ3 results) and reduce the grow-with-data evaluation (RQ4) to a descriptive comparison of model parameters before and after accumulation, rather than a full Bayesian network implementation.

References

[1] U.S. Department of Defense, "Procedures for Performing a Failure Mode, Effects and Criticality Analysis," *MIL-STD-1629A*, 1980.

[2] A. Aamodt and E. Plaza, "Case-based reasoning: Foundational issues, methodological variations, and system approaches," *AI Communications*, vol. 7, no. 1, pp. 39–59, 1994.

[3] J. de Kleer and B. C. Williams, "Diagnosing multiple faults," *Artificial Intelligence*, vol. 32, no. 1, pp. 97–130, 1987, doi: 10.1016/0004-3702(87)90063-4.

[4] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Mateo, CA: Morgan Kaufmann, 1988.

[5] G. Bellala, J. Stanley, S. K. Bhavnani, and C. Scott, "A rank-based approach to active diagnosis," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 9, pp. 2078–2090, 2013, doi: 10.1109/TPAMI.2013.30.

[6] S. Siddiqi and J. Huang, "Sequential diagnosis by abstraction," *Journal of Artificial Intelligence Research*, vol. 41, pp. 329–365, 2011, doi: 10.1613/jair.3296.

[7] A. Feldman, G. Provan, and A. J. C. van Gemund, "A framework and algorithm for model-based active testing," in *Proceedings of the First International Conference on Prognostics and Health Management (PHM'08)*, 2008, pp. 1–8.

[8] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3–4, pp. 379–423 and 623–656, 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.

[9] M. Natu and A. S. Sethi, "Probabilistic fault diagnosis using adaptive probing," in *Managing Virtualization of Networks and Services (DSOM 2007)*, Lecture Notes in Computer Science, vol. 4785, pp. 38–49, 2007.

[10] V. Lín, "Binary integer programming solution for troubleshooting with dependent actions," *Kybernetika*, vol. 53, no. 3, pp. 493–512, 2017, doi: 10.14736/kyb-2017-3-0493.

[11] Y. Liang, M. Xiao, X. Wang, X. Tang, H. Zhu, and J. Li, "A POMDP-based optimization method for sequential diagnostic strategy with unreliable tests," *IEEE Access*, vol. 7, pp. 75389–75397, 2019, doi: 10.1109/ACCESS.2019.2918867.

[12] A. X. Zheng, I. Rish, and A. Beygelzimer, "Efficient test selection in active diagnosis via entropy approximation," in *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI)*, pp. 675–682, 2005.

[13] E. van Gerwen, T. Heskes, and T. Claassen, "Efficient differential diagnosis using cost-aware active diagnosis," *International Journal of Prognostics and Health Management*, vol. 15, no. 1, 2024.

Task	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**16**
Schema + logging design (M1)	X	X														
Data structuring + NLP mapping (M2)		X	X	X												
Evaluation framework (metrics/splits/scripts)		X	X													
Baselines implementation + evaluation (M3)			X	X	X											
RQ1 ranking model + evaluation (M4)					X	X	X									
RQ2 updating + entropy tracking (M4)						X	X	X	X							
RQ3 action selection + cost comparison (M5)								X	X	X	X	X				
Grow-with-data evaluation / RQ4 (M6)											X	X	X			
Thesis writing (results/discussion/methods) (M6)												X	X	X	X	X
Final revision + submission prep (M6)														X	X	X
Milestones delivered	M1		M2		M3		M4			M5			M6			

Fig. 2. Gantt-style plan (Weeks 1–16 after the preparation report). Milestones indicate target completion points.

[14] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proceedings of the 2017 IEEE International Conference on Web Services (ICWS)*, pp. 33–40, 2017, doi: 10.1109/ICWS.2017.13.

[15] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, vol. 2, pp. 427–431, 2017.

[16] H. L. Semigran, J. A. Linder, C. Gidengil, and A. Mehrotra, "Evaluation of symptom checkers for self diagnosis and triage: audit study," *BMJ*, vol. 351, h3480, 2015, doi: 10.1136/bmj.h3480.

[17] I. Rish, "An empirical study of the naive Bayes classifier," in *Proceedings of the IJCAI Workshop on Empirical Methods in Artificial Intelligence*, pp. 41–46, 2001.